

SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

Particular	Details
Name:	Tharun Kumar.M
Reg no:	192571017
Department:	CSE&BIO
Programme:	B.Tech
Course Code & Course Name	CSA05 — Database Management Systems
Faculty Name	Dr.Kesavan
Assignment Title	National Vocational Certification and Exam Slot-Booking System: An Integrated Study in File Organization, Indexing, Hashing, Query Optimization, and Transaction Management
Date of Issue	02-09-2026
Date of Submission	02-09-2026
Maximum Marks	100
Course Outcome(s) – CO	CO4: Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. CO5: Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyze
SDG Mapping (SDG 1-17)	SDG 4 – Quality Education SDG 8 – Decent Work and Economic Growth SDG 9 – Industry, Innovation and Infrastructure

Working Module

SQL Database Implementation

The following implementation uses MySQL 8.0 and InnoDB. The screenshots are monochrome, MySQL-command-line-style illustrations. They are intended to show the required evidence format; the final submission should replace illustrative execution messages with screenshots captured from the student's own MySQL session.

SQL Implementation 1: Create the database

SQL Query:

CREATE DATABASE vocational_exam;

MySQL Command Line Output:

```
MySQL 8.0 Command Line Client
mysql> CREATE DATABASE vocational_exam;

Query OK, 1 row affected (0.01 sec)
```

The database provides the logical container for the certification system. The command is expected to succeed when the connected MySQL account has permission to create databases.

SQL Implementation 2: Select the database

SQL Query:

USE vocational_exam;

MySQL Command Line Output:

```
MySQL 8.0 Command Line Client
mysql> USE vocational_exam;

Database changed
```

The USE statement makes vocational_exam the active database for subsequent table and query operations.

11.1 Create Tables

**CREATE TABLE candidates (
 candidate_id INT PRIMARY KEY,
 candidate_name VARCHAR(100) NOT NULL,**

```
    phone VARCHAR(15) NOT NULL UNIQUE,  
    email VARCHAR(120) NOT NULL UNIQUE,  
    city VARCHAR(60) NOT NULL  
  ) ENGINE=InnoDB;
```

```
CREATE TABLE exam_centers (  
    center_id INT PRIMARY KEY,  
    center_name VARCHAR(120) NOT NULL,  
    city VARCHAR(60) NOT NULL,  
    center_capacity INT NOT NULL CHECK (center_capacity > 0)  
  ) ENGINE=InnoDB;
```

```
CREATE TABLE exam_slots (  
    slot_id INT PRIMARY KEY,  
    center_id INT NOT NULL,  
    exam_date DATE NOT NULL,  
    start_time TIME NOT NULL,  
    end_time TIME NOT NULL,  
    capacity INT NOT NULL CHECK (capacity > 0),  
    booked_count INT NOT NULL DEFAULT 0,  
    status VARCHAR(15) NOT NULL DEFAULT 'OPEN',  
    FOREIGN KEY (center_id) REFERENCES exam_centers(center_id),  
    CHECK (booked_count >= 0 AND booked_count <= capacity),  
    CHECK (status IN ('OPEN','CLOSED'))  
  ) ENGINE=InnoDB;
```

```
CREATE TABLE bookings (  
    booking_id INT AUTO_INCREMENT PRIMARY KEY,  
    candidate_id INT NOT NULL,  
    slot_id INT NOT NULL,  
    status VARCHAR(15) NOT NULL DEFAULT 'BOOKED',  
    booked_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (candidate_id) REFERENCES candidates(candidate_id),  
    FOREIGN KEY (slot_id) REFERENCES exam_slots(slot_id),  
    UNIQUE (candidate_id, slot_id),  
    CHECK (status IN ('BOOKED','CANCELLED'))  
  ) ENGINE=InnoDB;
```

```
CREATE TABLE exam_attempts (  
    attempt_id BIGINT PRIMARY KEY,
```

```
candidate_id INT NOT NULL,  
center_id INT NOT NULL,  
exam_date DATE NOT NULL,  
responses_json JSON,  
start_time DATETIME,  
end_time DATETIME,  
score DECIMAL(6,2) CHECK (score >= 0 AND score <= 100),  
FOREIGN KEY (candidate_id) REFERENCES candidates(candidate_id),  
FOREIGN KEY (center_id) REFERENCES exam_centers(center_id)  
) ENGINE=InnoDB;
```

```
CREATE TABLE attendance (  
attendance_id BIGINT PRIMARY KEY,  
attempt_id BIGINT NOT NULL,  
attendance_status VARCHAR(15) NOT NULL,  
check_in_time DATETIME,  
FOREIGN KEY (attempt_id) REFERENCES exam_attempts(attempt_id),  
CHECK (attendance_status IN ('PRESENT','ABSENT'))  
) ENGINE=InnoDB;
```

```
CREATE TABLE results (  
result_id BIGINT PRIMARY KEY,  
attempt_id BIGINT NOT NULL UNIQUE,  
candidate_id INT NOT NULL,  
score DECIMAL(6,2) NOT NULL CHECK (score >= 0 AND score <= 100),  
percentage DECIMAL(6,2) NOT NULL CHECK (percentage >= 0 AND percentage <=  
100),  
result_status VARCHAR(15) NOT NULL DEFAULT 'DRAFT',  
published_at DATETIME NULL,  
FOREIGN KEY (attempt_id) REFERENCES exam_attempts(attempt_id),  
FOREIGN KEY (candidate_id) REFERENCES candidates(candidate_id),  
CHECK (result_status IN ('DRAFT','PUBLISHED'))  
) ENGINE=InnoDB;
```

```
MySQL 8.0 Command Line: X + v
mysql> CREATE TABLE candidates (
->   candidate_id INT PRIMARY KEY,
->   candidate_name VARCHAR(100) NOT NULL,
->   phone VARCHAR(15) NOT NULL UNIQUE,
->   email VARCHAR(120) NOT NULL UNIQUE,
->   city VARCHAR(60) NOT NULL
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.13 sec)

mysql>
mysql> CREATE TABLE exam_centers (
->   center_id INT PRIMARY KEY,
->   center_name VARCHAR(120) NOT NULL,
->   city VARCHAR(60) NOT NULL,
->   center_capacity INT NOT NULL CHECK (center_capacity > 0)
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE TABLE exam_slots (
->   slot_id INT PRIMARY KEY,
->   center_id INT NOT NULL,
->   exam_date DATE NOT NULL,
->   start_time TIME NOT NULL,
->   end_time TIME NOT NULL,
->   capacity INT NOT NULL CHECK (capacity > 0),
->   booked_count INT NOT NULL DEFAULT 0,
->   status VARCHAR(15) NOT NULL DEFAULT 'OPEN',
->   FOREIGN KEY (center_id) REFERENCES exam_centers(center_id),
->   CHECK (booked_count >= 0 AND booked_count <= capacity),
->   CHECK (status IN ('OPEN', 'CLOSED'))
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.07 sec)

mysql>
```

```
MySQL 8.0 Command Line: X + v
mysql>
mysql> CREATE TABLE bookings (
->   booking_id INT AUTO_INCREMENT PRIMARY KEY,
->   candidate_id INT NOT NULL,
->   slot_id INT NOT NULL,
->   status VARCHAR(15) NOT NULL DEFAULT 'BOOKED',
->   booked_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
->   FOREIGN KEY (candidate_id) REFERENCES candidates(candidate_id),
->   FOREIGN KEY (slot_id) REFERENCES exam_slots(slot_id),
->   UNIQUE (candidate_id, slot_id),
->   CHECK (status IN ('BOOKED', 'CANCELLED'))
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> CREATE TABLE exam_attempts (
->   attempt_id BIGINT PRIMARY KEY,
->   candidate_id INT NOT NULL,
->   center_id INT NOT NULL,
->   exam_date DATE NOT NULL,
->   responses_json JSON,
->   start_time DATETIME,
->   end_time DATETIME,
->   score DECIMAL(6,2) CHECK (score >= 0 AND score <= 100),
->   FOREIGN KEY (candidate_id) REFERENCES candidates(candidate_id),
->   FOREIGN KEY (center_id) REFERENCES exam_centers(center_id)
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> CREATE TABLE attendance (
->   attendance_id BIGINT PRIMARY KEY,
->   attempt_id BIGINT NOT NULL,
->   attendance_status VARCHAR(15) NOT NULL,
->   check_in_time DATETIME,
```

```
MySQL 8.0 Command Line Client
-> end_time DATETIME,
-> score DECIMAL(6,2) CHECK (score >= 0 AND score <= 100),
-> FOREIGN KEY (candidate_id) REFERENCES candidates(candidate_id),
-> FOREIGN KEY (center_id) REFERENCES exam_centers(center_id)
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> CREATE TABLE attendance (
-> attendance_id BIGINT PRIMARY KEY,
-> attempt_id BIGINT NOT NULL,
-> attendance_status VARCHAR(15) NOT NULL,
-> check_in_time DATETIME,
-> FOREIGN KEY (attempt_id) REFERENCES exam_attempts(attempt_id),
-> CHECK (attendance_status IN ('PRESENT', 'ABSENT'))
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> CREATE TABLE results (
-> result_id BIGINT PRIMARY KEY,
-> attempt_id BIGINT NOT NULL UNIQUE,
-> candidate_id INT NOT NULL,
-> score DECIMAL(6,2) NOT NULL CHECK (score >= 0 AND score <= 100),
-> percentage DECIMAL(6,2) NOT NULL CHECK (percentage >= 0 AND percentage <= 100),
-> result_status VARCHAR(15) NOT NULL DEFAULT 'DRAFT',
-> published_at DATETIME NULL,
-> FOREIGN KEY (attempt_id) REFERENCES exam_attempts(attempt_id),
-> FOREIGN KEY (candidate_id) REFERENCES candidates(candidate_id),
-> CHECK (result_status IN ('DRAFT', 'PUBLISHED'))
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql>
```

```
MySQL 8.0 Command Line Client

mysql> CREATE TABLE candidates (...);

mysql> CREATE TABLE exam_centers (...);

mysql> CREATE TABLE exam_slots (...);

mysql> CREATE TABLE exam_attempts (...);

mysql> CREATE TABLE bookings (...);

mysql> CREATE TABLE results (...);

Query OK, 0 rows affected (0.02 sec)
Query OK, 0 rows affected (0.03 sec)
Query OK, 0 rows affected (0.02 sec)
Query OK, 0 rows affected (0.03 sec)
Query OK, 0 rows affected (0.02 sec)
Query OK, 0 rows affected (0.02 sec)
```

The schema establishes primary keys for entity identification, foreign keys for referential integrity, uniqueness rules for duplicate prevention, and CHECK constraints for valid numeric and status values. InnoDB is selected because transactional consistency and row-level locking are central requirements.

11.2 Insert Representative Data

INSERT INTO candidates(candidate_id,candidate_name,phone,email,city) VALUES (1001,'Tharun Kumar','9876543210','tharun@example.com','Chennai'),

```
(1002,'Arun Kumar','9876543211','arun@example.com','Chennai'),  
(1003,'Priya N','9876543212','priya@example.com','Coimbatore');
```

```
INSERT INTO exam_centers(center_id,center_name,city,center_capacity) VALUES  
(201,'Chennai Central','Chennai',200),  
(202,'Coimbatore Hub','Coimbatore',150);
```

```
INSERT INTO exam_slots(slot_id,center_id,exam_date,start_time,end_time,capacity)  
VALUES  
(101,201,'2026-09-15','09:00:00','12:00:00',2),  
(102,201,'2026-09-15','14:00:00','17:00:00',2),  
(103,202,'2026-09-16','14:00:00','17:00:00',3);
```

```
mysql> INSERT INTO candidates(candidate_id,candidate_name,phone,email,city) VALUES  
-> (1001,'Tharun Kumar','9876543210','tharun@example.com','Chennai'),  
-> (1002,'Arun Kumar','9876543211','arun@example.com','Chennai'),  
-> (1003,'Priya N','9876543212','priya@example.com','Coimbatore');  
ERROR 1062 (23000): Duplicate entry '1001' for key 'candidates.PRIMARY'  
mysql>  
mysql> INSERT INTO exam_centers(center_id,center_name,city,center_capacity) VALUES  
-> (201,'Chennai Central','Chennai',200),  
-> (202,'Coimbatore Hub','Coimbatore',150);  
Query OK, 2 rows affected (0.02 sec)  
Records: 2 Duplicates: 0 Warnings: 0  
  
mysql>  
mysql> INSERT INTO exam_slots(slot_id,center_id,exam_date,start_time,end_time,capacity) VALUES  
-> (101,201,'2026-09-15','09:00:00','12:00:00',2),  
-> (102,201,'2026-09-15','14:00:00','17:00:00',2),  
-> (103,202,'2026-09-16','14:00:00','17:00:00',3);  
Query OK, 3 rows affected (0.01 sec)  
Records: 3 Duplicates: 0 Warnings: 0  
  
mysql>
```

MySQL 8.0 Command Line Client

```
mysql> INSERT INTO candidates VALUES  
  
mysql> (1001,'Tharun Kumar','9876543210','tharun@example.com'),  
  
mysql> (1002,'Arun Kumar','9876543211','arun@example.com'),  
  
mysql> (1003,'Priya N','9876543212','priya@example.com');  
  
Query OK, 3 rows affected (0.01 sec)  
Records: 3 Duplicates: 0 Warnings: 0
```

SQL Implementation 3: Display candidates in sorted order

SQL Query:

```
SELECT candidate_id, candidate_name, city  
FROM candidates ORDER BY candidate_id;
```

MySQL Command Line Output:

```
MySQL 8.0 Command Line Client
mysql> SELECT candidate_id, candidate_name, city
mysql> FROM candidates ORDER BY candidate_id;

+-----+-----+-----+
| candidate_id | candidate_name | city      |
+-----+-----+-----+
| 1001         | Tharun Kumar   | Chennai   |
| 1002         | Arun Kumar     | Chennai   |
| 1003         | Priya N        | Coimbatore|
+-----+-----+-----+
3 rows in set (0.00 sec)
```

ORDER BY provides deterministic presentation of candidate records. This also demonstrates a basic retrieval operation on the relational schema.

11.3 Index Creation

```
CREATE INDEX idx_attempt_candidate
ON exam_attempts(candidate_id);
```

```
CREATE INDEX idx_attempt_center_date
ON exam_attempts(center_id, exam_date);
```

```
CREATE INDEX idx_slot_center_date
ON exam_slots(center_id, exam_date, start_time);
```

```
CREATE INDEX idx_booking_candidate
ON bookings(candidate_id, status);
```

```
CREATE INDEX idx_result_candidate
ON results(candidate_id, result_status);
```


MySQL 8.0 Command Line Client

```
mysql> CREATE INDEX idx_attempt_candidate ON exam_attempts(candidate_id);

mysql> CREATE INDEX idx_attempt_center_date ON exam_attempts(center_id, exam_date);

mysql> CREATE INDEX idx_slot_center_date ON exam_slots(center_id, exam_date, start_time);

Query OK, 0 rows affected (0.04 sec)
Query OK, 0 rows affected (0.05 sec)
Query OK, 0 rows affected (0.03 sec)
```

The index definitions are chosen around the access patterns specified in the assignment. The candidate index supports history retrieval, the center/date index supports audit queries, and the slot index supports scheduling and availability searches.

SQL Implementation 4: Retrieve candidate attempt history

SQL Query:

**CREATE INDEX idx_attempt_candidate
ON exam_attempts(candidate_id);**

**CREATE INDEX idx_attempt_center_date
ON exam_attempts(center_id, exam_date);**

**CREATE INDEX idx_slot_center_date
ON exam_slots(center_id, exam_date, start_time);**

**CREATE INDEX idx_booking_candidate
ON bookings(candidate_id, status);**

**CREATE INDEX idx_result_candidate
ON results(candidate_id, result_status);**

MySQL Command Line Output:

MySQL 8.0 Command Line Client

```
mysql> SELECT a.attempt_id, a.candidate_id, c.candidate_name,  
  
mysql>          a.center_id, a.exam_date, a.score  
  
mysql> FROM exam_attempts a  
  
mysql> JOIN candidates c ON c.candidate_id = a.candidate_id  
  
mysql> WHERE a.candidate_id = 1001  
  
mysql> ORDER BY a.exam_date DESC;
```

```
+-----+-----+-----+-----+-----+-----+  
| attempt_id | candidate_id | candidate_name | center_id | exam_date | score |  
+-----+-----+-----+-----+-----+-----+  
| 5003      | 1001        | Tharun Kumar  | 201       | 2026-08-20 | 88.00 |  
| 5001      | 1001        | Tharun Kumar  | 201       | 2026-07-15 | 82.00 |  
+-----+-----+-----+-----+-----+-----+  
2 rows in set (0.00 sec)
```

The candidate identifier is highly selective and is supported by `idx_attempt_candidate`. The query joins candidate information only after identifying the relevant attempts.

SQL Implementation 5: Perform a center-wise audit

SQL Query:

```
SELECT center_id, exam_date, COUNT(*) AS attempts,  
       AVG(score) AS average_score  
FROM exam_attempts  
WHERE center_id = 201  
       AND exam_date = '2026-08-20'  
GROUP BY center_id, exam_date;
```

MySQL Command Line Output:

MySQL 8.0 Command Line Client

```
mysql> SELECT center_id, exam_date, COUNT(*) AS attempts,  
mysql>          AVG(score) AS average_score  
mysql> FROM exam_attempts  
mysql> WHERE center_id = 201 AND exam_date = '2026-08-20'  
mysql> GROUP BY center_id, exam_date;
```

```
+-----+-----+-----+-----+  
| center_id | exam_date | attempts | average_score |  
+-----+-----+-----+-----+  
| 201       | 2026-08-20 | 3       | 81.67         |  
+-----+-----+-----+-----+  
1 row in set (0.00 sec)
```

The composite center/date index matches the two filtering columns and allows the DBMS to narrow the candidate rows before performing the aggregate calculation.

SQL Implementation 6: Generate a booking report using joins

SQL Query:

```
SELECT c.candidate_name, ec.center_name,  
       s.exam_date, s.start_time, b.status  
FROM bookings b  
JOIN candidates c ON c.candidate_id=b.candidate_id  
JOIN exam_slots s ON s.slot_id=b.slot_id  
JOIN exam_centers ec ON ec.center_id=s.center_id  
ORDER BY s.exam_date, s.start_time;
```

MySQL Command Line Output:

```
MySQL 8.0 Command Line Client
mysql> SELECT c.candidate_name, ec.center_name,
mysql>          s.exam_date, s.start_time, b.status
mysql> FROM bookings b
mysql> JOIN candidates c ON c.candidate_id=b.candidate_id
mysql> JOIN exam_slots s ON s.slot_id=b.slot_id
mysql> JOIN exam_centers ec ON ec.center_id=s.center_id
mysql> ORDER BY s.exam_date, s.start_time;

+-----+-----+-----+-----+-----+
| candidate_name | center_name | exam_date | start_time | status |
+-----+-----+-----+-----+-----+
| Tharun Kumar  | Chennai Central | 2026-09-15 | 09:00:00 | BOOKED |
| Arun Kumar    | Chennai Central | 2026-09-15 | 09:00:00 | BOOKED |
| Priya N       | Coimbatore Hub | 2026-09-16 | 14:00:00 | BOOKED |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

The query demonstrates multi-table joins and produces a user-oriented booking report without duplicating candidate, center, and slot data in a single table.

SQL Implementation 7: Use a subquery for high-performing candidates

SQL Query:

```
SELECT candidate_id, candidate_name
FROM candidates
WHERE candidate_id IN (
    SELECT candidate_id
    FROM results
    WHERE percentage >= 80
);
```

MySQL Command Line Output:

```
MySQL 8.0 Command Line Client
mysql> SELECT candidate_id, candidate_name
mysql> FROM candidates
mysql> WHERE candidate_id IN (
mysql>     SELECT candidate_id FROM results WHERE percentage >= 80
mysql> );

+-----+-----+
| candidate_id | candidate_name |
+-----+-----+
| 1001         | Tharun Kumar   |
| 1003         | Priya N        |
+-----+-----+
2 rows in set (0.00 sec)
```

The subquery identifies candidate identifiers meeting the performance condition, after which the outer query retrieves the corresponding candidate names.

12. Query Processing

A DBMS processes an SQL request through parsing and validation, logical query transformation, optimization, physical access-path selection, and execution. The optimizer considers available indexes, estimated row counts, join methods, and filtering conditions. In this project, the important optimization objective is to avoid full scans of the large exam-attempt archive and to keep booking/result transactions short.

For candidate history, the optimizer can use `idx_attempt_candidate` to locate the target candidate's rows. For a center/date audit, `idx_attempt_center_date` supports a two-column lookup or range access. For booking, the primary key on `slot_id` makes the locked slot lookup selective, while the candidate-slot uniqueness rule provides an additional integrity barrier.

13. Query Optimization and Execution Plan Analysis

The following example compares an illustrative execution plan before and after creating the center/date composite index. The row counts shown in the terminal images are examples for demonstration and are not claimed as measurements from a particular machine.

13.1 Before Optimization

EXPLAIN

SELECT *

FROM exam_attempts

WHERE center_id = 201

AND exam_date = '2026-08-20';

MySQL 8.0 Command Line Client								
mysql> EXPLAIN SELECT * FROM exam_attempts								
mysql> WHERE center_id = 201 AND exam_date = '2026-08-20';								
+-----+-----+-----+-----+-----+-----+-----+-----+-----+								
id	select_type	table	type	possible_keys	key	rows	Extra	
+-----+-----+-----+-----+-----+-----+-----+-----+-----+								
1	SIMPLE	exam_attempts	ALL	NULL	NULL	1000000	Using where	
+-----+-----+-----+-----+-----+-----+-----+-----+-----+								
1 row in set (0.00 sec)								
Illustrative plan before indexing								

The illustrative plan uses ALL access and shows a full table scan. If the archive contains millions of records, examining every row for a selective center/date condition is unnecessarily expensive.

13.2 After Optimization

CREATE INDEX idx_attempt_center_date

ON exam_attempts(center_id, exam_date);

EXPLAIN

SELECT *

FROM exam_attempts

WHERE center_id = 201

AND exam_date = '2026-08-20';

```

MySQL 8.0 Command Line Client

mysql> CREATE INDEX idx_attempt_center_date
mysql> ON exam_attempts(center_id, exam_date);

mysql> EXPLAIN SELECT * FROM exam_attempts
mysql> WHERE center_id = 201 AND exam_date = '2026-08-20';

+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+
| id | select_type | table          | type | possible_keys          | key              | rows |
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+
| 1  | SIMPLE      | exam_attempts | ref  | idx_attempt_center_date | idx_attempt_center_date | 120 |
index condition |
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+
1 row in set (0.00 sec)
Illustrative plan after indexing

```

The indexed plan demonstrates reference access through the composite index. The optimizer can identify the relevant center/date entries without scanning the complete archive. The exact plan and row estimate must be verified using EXPLAIN on the student's actual dataset.

Query	Before Optimization	After Optimization	Improvement
Center/date audit	Illustrative full scan	Illustrative index reference access	Fewer rows examined
Candidate history	Illustrative full scan	Candidate index access	Selective lookup
Slot search	Broad filtering	Composite slot index	Faster availability lookup

14. Transaction Management

Transaction management is required because slot booking and result publishing consist of multiple dependent operations. A booking is not complete merely because a seat counter changed; the booking row and the seat count must represent the same outcome. Likewise, a result should not be partially updated. ACID properties provide the required model: atomicity keeps related changes together, consistency preserves constraints, isolation controls concurrent interaction, and durability preserves committed changes.

14.1 Successful Slot Booking with COMMIT

START TRANSACTION;

```
MySQL 8.0 Command Line Client
mysql> START TRANSACTION;

mysql> SELECT capacity, booked_count FROM exam_slots

mysql> WHERE slot_id=101 FOR UPDATE;

mysql> UPDATE exam_slots SET booked_count=booked_count+1

mysql> WHERE slot_id=101 AND booked_count < capacity;

mysql> INSERT INTO bookings(candidate_id,slot_id,status)

mysql> VALUES(1001,101,'BOOKED');

mysql> COMMIT;

Query OK, 0 rows affected (0.00 sec)
1 row in set (0.00 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 0 rows affected (0.01 sec)
Booking committed successfully
```

SELECT ... FOR UPDATE locks the selected slot row until the transaction ends. The application should check the UPDATE row count before inserting the booking. If the update affects zero rows, the slot is full or invalid and the transaction should be rolled back.

14.2 Failed Booking with ROLLBACK

START TRANSACTION;

**SELECT capacity, booked_count
FROM exam_slots
WHERE slot_id = 102
FOR UPDATE;**

**UPDATE exam_slots
SET booked_count = booked_count + 1
WHERE slot_id = 102
AND booked_count < capacity;**

-- If capacity is unavailable:
ROLLBACK;

MySQL 8.0 Command Line Client

```
mysql> START TRANSACTION;

mysql> SELECT capacity, booked_count FROM exam_slots

mysql> WHERE slot_id=102 FOR UPDATE;

mysql> UPDATE exam_slots SET booked_count=booked_count+1

mysql> WHERE slot_id=102 AND booked_count < capacity;

mysql> ROLLBACK;

Query OK, 0 rows affected (0.00 sec)
1 row in set (0.00 sec)
Query OK, 0 rows affected (0.01 sec)
Query OK, 0 rows affected (0.00 sec)
Query OK, 0 rows affected (0.01 sec)
Transaction rolled back; no partial booking retained
```

ROLLBACK removes the transaction's uncommitted changes. This prevents a partial state in which the slot count changes but no valid booking exists.

14.3 SAVEPOINT Demonstration

START TRANSACTION;

UPDATE candidates

SET phone = '9000000001'

WHERE candidate_id = 1001;

SAVEPOINT candidate_update;

UPDATE candidates

SET email = 'updated@example.com'

WHERE candidate_id = 1001;

ROLLBACK TO SAVEPOINT candidate_update;

COMMIT;

MySQL 8.0 Command Line Client

```
mysql> START TRANSACTION;

mysql> UPDATE candidates SET phone='9000000001' WHERE candidate_id=1001;

mysql> SAVEPOINT candidate_update;

mysql> UPDATE candidates SET email='updated@example.com' WHERE candidate_id=1001;

mysql> ROLLBACK TO SAVEPOINT candidate_update;

mysql> COMMIT;

Query OK, 0 rows affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 0 rows affected (0.00 sec)
Query OK, 1 row affected (0.01 sec)
Query OK, 0 rows affected (0.00 sec)
Query OK, 0 rows affected (0.01 sec)
Phone update committed; email update rolled back to savepoint
```

SAVEPOINT allows part of a longer transaction to be undone without discarding earlier successful work. Here, the phone update remains eligible for commit while the email change is rolled back to the savepoint.

15. Concurrency Control

The main concurrency hazards are double booking and lost result updates. A double-booking race occurs when two sessions read the same remaining capacity before either session updates it. Row locking prevents both sessions from independently changing the same slot state. A lost update occurs when two examiners read the same result value and then write different values without coordination. Locking the result row before modification prevents the later transaction from silently overwriting an earlier committed update.

15.1 Recommended Isolation and Locking Strategy

- Use InnoDB tables for transactional locking.
- Use a short transaction around each booking operation.
- Lock the selected slot row with `SELECT ... FOR UPDATE` before changing capacity.
- Use a `UNIQUE(candidate_id, slot_id)` constraint to prevent duplicate candidate bookings.
- Lock a result row before changing its score when concurrent editors can target the same result.
- Use `REPEATABLE READ` as a practical MySQL default and explicit row locks for the critical write paths; `SERIALIZABLE` can be considered where stronger protection is justified.

The choice balances consistency with throughput. Stronger isolation can increase waiting and contention, so the design uses explicit row locking only where it is necessary for the critical state transition.

15.2 Result Publishing Transaction

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
START TRANSACTION;
```

```
SELECT score  
FROM results  
WHERE result_id = 9001  
FOR UPDATE;
```

```
UPDATE results  
SET score = 88,  
    percentage = 88,  
    result_status = 'PUBLISHED',  
    published_at = NOW()  
WHERE result_id = 9001;
```

```
COMMIT;
```

```
MySQL 8.0 Command Line Client  
mysql> SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
  
mysql> START TRANSACTION;  
  
mysql> SELECT score FROM results WHERE result_id=9001 FOR UPDATE;  
  
mysql> UPDATE results SET score=88, percentage=88 WHERE result_id=9001;  
  
mysql> COMMIT;  
  
Query OK, 0 rows affected (0.00 sec)  
Query OK, 0 rows affected (0.00 sec)  
1 row in set (0.00 sec)  
Query OK, 1 row affected (0.01 sec)  
Query OK, 0 rows affected (0.01 sec)  
Result update completed without an unlocked lost update
```

The row lock serializes competing updates to the same result. The transaction should validate the examiner's authorization and score range before committing.

16. Hash Lookup Demonstration

-- Bucket calculation

```
SELECT candidate_id,  
       MOD(candidate_id, 101) AS bucket_no  
FROM candidates  
WHERE candidate_id = 1001;
```

-- Example result:

-- candidate_id = 1001

-- bucket_no = 91

This represents the logical static-hashing method. In a production implementation, the application can maintain a bucket directory or a generated bucket column. Collision handling uses chaining, so multiple candidate identifiers can reside in the same bucket without losing records.

17. Query Design for Booking and Results

-- Available slots at a center on a date

```
SELECT slot_id, start_time, end_time,  
       capacity - booked_count AS available_seats  
FROM exam_slots  
WHERE center_id = 201  
      AND exam_date = '2026-09-15'  
      AND status = 'OPEN'  
      AND booked_count < capacity  
ORDER BY start_time;
```

-- Candidate result report

```
SELECT c.candidate_id, c.candidate_name,  
       r.score, r.percentage, r.result_status  
FROM candidates c  
LEFT JOIN results r  
      ON r.candidate_id = c.candidate_id  
ORDER BY c.candidate_id;
```

-- Center performance summary

```
SELECT ec.center_id, ec.center_name,  
       COUNT(a.attempt_id) AS total_attempts,  
       AVG(a.score) AS average_score  
FROM exam_centers ec  
LEFT JOIN exam_attempts a
```

```
ON a.center_id = ec.center_id  
GROUP BY ec.center_id, ec.center_name  
HAVING COUNT(a.attempt_id) > 0;
```

These queries demonstrate filtering, ordering, LEFT JOIN, aggregate functions, GROUP BY, and HAVING. The selected indexes should be verified with EXPLAIN to confirm that the optimizer is using them on the actual dataset.

18. Security and Privacy

Candidate personal details, responses, scores, and published results are sensitive application data. Database privileges should therefore follow least-privilege principles. Auditors may receive read-only access to the tables they require, while booking services and result processors should receive only the permissions needed for their tasks.

18.1 GRANT and REVOKE Example

```
CREATE USER 'exam_auditor'@'localhost'  
IDENTIFIED BY 'StrongPass!26';
```

```
GRANT SELECT  
ON vocational_exam.*  
TO 'exam_auditor'@'localhost';
```

```
REVOKE SELECT  
ON vocational_exam.results  
FROM 'exam_auditor'@'localhost';
```

```
MySQL 8.0 Command Line Client  
mysql> CREATE USER 'exam_auditor'@'localhost' IDENTIFIED BY 'StrongPass!26';  
  
mysql> GRANT SELECT ON vocational_exam.* TO 'exam_auditor'@'localhost';  
  
mysql> REVOKE SELECT ON vocational_exam.results FROM 'exam_auditor'@'localhost';  
  
Query OK, 0 rows affected (0.03 sec)  
Query OK, 0 rows affected (0.01 sec)  
Query OK, 0 rows affected (0.01 sec)
```

The password shown is an example and should not be reused in a real deployment. Actual security configuration should use strong secrets, secure credential storage, role-based privileges, auditing, and encrypted connections where appropriate.